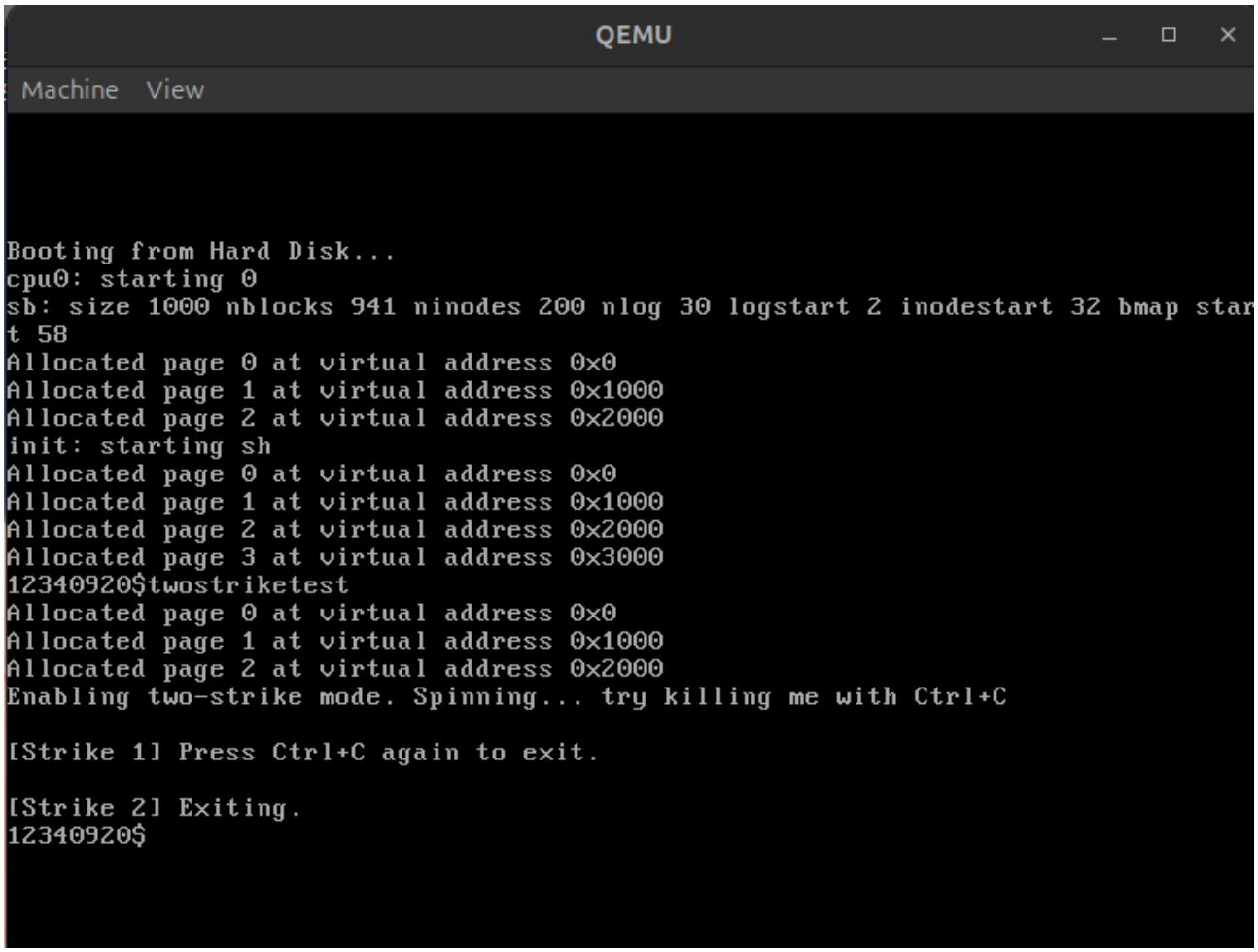
QUE 1: XV6

Screenshot:



File to change:

1. proc.h

```
int twostrike_mode;
 int strike_count;
```

2. proc.c (in allocproc())

```
// kernel/proc.c, inside allocproc() after other initializations
p->twostrike_mode = 0;
p->strike_count = 0;
```

3. console.c

```
void
consoleintr(int (*getc)(void))
{
  int c, doprocdump = 0,do_twostrike=0;
```

```
acquire(&cons.lock);
while((c = getc()) >= 0){
  switch(c){
  case 3:
    do_twostrike = 1;
  break;
  case C('P'):  // Process listing.
    // procdump() locks cons.lock indirectly; invoke later
    doprocdump = 1;
    break;
  case C('U'):  // Kill line.
    while(input.e != input.w &&
          input.buf[(input.e-1) % INPUT_BUF] != '\n'){
      input.e--;
      consputc(BACKSPACE);
    }
    break;
  case C('H'): case '\x7f':  // Backspace
    if(input.e != input.w){
      input.e--;
      consputc(BACKSPACE);
    }
    break;
  default:
    if(c != 0 && input.e-input.r < INPUT_BUF){
      c = (c == '\r') ? '\n' : c;
      input.buf[input.e++ % INPUT_BUF] = c;
      consputc(c);
      if(c == '\n' || c == C('D') || input.e == input.r+INPUT_BUF){
        input.w = input.e;
        wakeup(&input.r);
      }
    }
    break;
  }
}
release(&cons.lock);
if(doprocdump) {
  procdump();  // now call procdump() wo. cons.lock held
}
if(do_twostrike) {
  struct proc *p = myproc();

  // Only consider if there's a current process and it is RUNNING
  if(p != 0 && p->state == RUNNING) {

    if(p->twostrike_mode == 1) {

      // If this is the FIRST strike (strike_count == 0)
      if(p->strike_count == 0) {

        // Mark that we've seen the first strike
```

```
          p->strike_count = 1;
          cprintf("\n[Strike 1] Press Ctrl+C again to exit.\n");

        } else {
          cprintf("\n[Strike 2] Exiting.\n");
          // Mark process killed (will be reaped as usual)
          p->killed = 1;
        }

      } else {
        // Not in two-strike mode, kill immediately (existing behavior)
        p->killed = 1;
      }
    }
  }
}
```

4. sysproc.c

```
int
sys_twostrike(void)
{
  int enabled;
  if(argint(0, &enabled) < 0)
    return -1;

  struct proc *p = myproc();
  if(p == 0)
    return -1;

  if (enabled)
    p->twostrike_mode = 1;
  else {
    p->twostrike_mode = 0;
    p->strike_count = 0; // reset any existing strike count when disabled
  }

  return 0;
}
```

5. syscall.h

```
#define SYS_twostrike 35
```

6. syscall.c

```
// at top with other externs
extern int sys_twostrike(void);

// inside the syscalls table
[SYS_twostrike]   sys_twostrike,
```

7. usys.S

SYSCALL(twostrike)

8. user.h

int twostrike(int enabled);

9. Create file: twostriketest.c

```
// user/twostriketest.c
#include "types.h"
#include "stat.h"
#include "user.h"

int
main(int argc, char *argv[])
{
  printf(1, "Enabling two-strike mode. Spinning... try killing me with Ctrl+C\n");

  // Call the system call to enable the feature
  twostrike(1);

  // Busy loop forever
  while(1) {
    // busy wait
  }

  exit();
}
```

10. MAKEFILE

_twostriketest

----------done----------

Que 2:

Code:

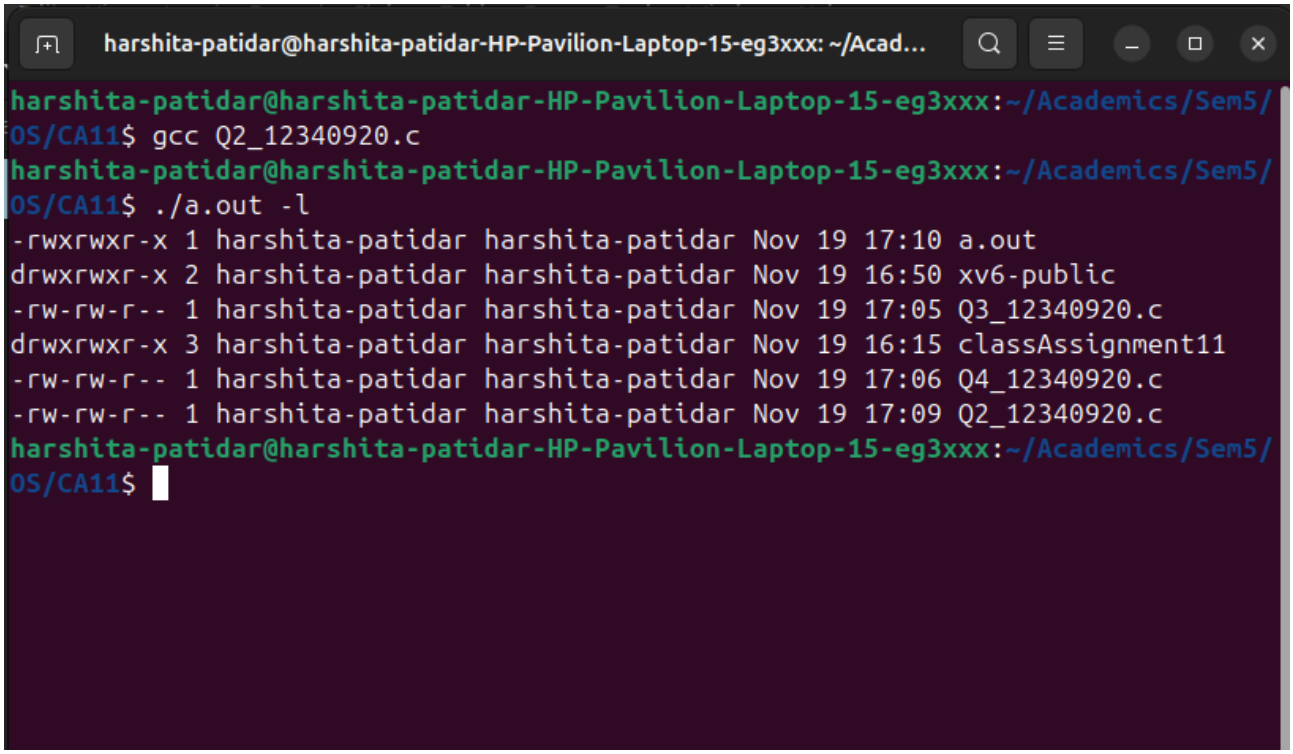
```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <dirent.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <pwd.h>
#include <grp.h>
#include <unistd.h>
#include <time.h>
```

```
void print_permissions(mode_t mode) {
    printf( (S_ISDIR(mode)) ? "d" : "-");
    printf( (mode & S_IRUSR) ? "r" : "-");
    printf( (mode & S_IWUSR) ? "w" : "-");
    printf( (mode & S_IXUSR) ? "x" : "-");
    printf( (mode & S_IRGRP) ? "r" : "-");
    printf( (mode & S_IWGRP) ? "w" : "-");
    printf( (mode & S_IXGRP) ? "x" : "-");
    printf( (mode & S_IROTH) ? "r" : "-");
    printf( (mode & S_IWOTH) ? "w" : "-");
    printf( (mode & S_IXOTH) ? "x" : "-");
}


void print_file_details(const char *path, const char *name) {
    char fullpath[1024];
    snprintf(fullpath, sizeof(fullpath), "%s/%s", path, name);

    struct stat sb;
    if (stat(fullpath, &sb) == -1) {
        perror("stat");
        return;
    }
        // Print file permissions
    print_permissions(sb.st_mode);
    printf(" ");

    // Number of hard links
    printf("%lu ", sb.st_nlink);

    // Owner username
    struct passwd *pw = getpwuid(sb.st_uid);
    if (pw)
        printf("%s ", pw->pw_name);
    else
        printf("%d ", sb.st_uid);

    // Group name
    struct group *gr = getgrgid(sb.st_gid);
    if (gr)
        printf("%s ", gr->gr_name);
    else
        printf("%d ", sb.st_gid);

    // Time format (like ls)
    char timebuf[64];
    struct tm *tm_info = localtime(&sb.st_mtime);
    strftime(timebuf, sizeof(timebuf), "%b %d %H:%M", tm_info);

    printf("%s ", timebuf);
```

```
    // File name
    printf("%s\n", name);

}

int main(int argc, char *argv[]) {
    int long_format = 0;
    char *dirpath = NULL;


    if (argc == 1) {
        dirpath = ".";
    } else if (argc == 2) {
        if (strcmp(argv[1], "-l") == 0)
            dirpath = ".";
        else
            dirpath = argv[1];
    } else if (argc == 3 && strcmp(argv[1], "-l") == 0) {
        long_format = 1;
        dirpath = argv[2];
    } else {
        fprintf(stderr, "Usage: %s [-l] [directory]\n", argv[0]);
        exit(1);
    }

    if (argc >= 2 && strcmp(argv[1], "-l") == 0)
        long_format = 1;

    DIR *dir = opendir(dirpath);
    if (!dir) {
        perror("opendir");
        return 1;
    }

    struct dirent *entry;
    while ((entry = readdir(dir)) != NULL) {
         if (strcmp(entry->d_name, ".") == 0 || strcmp(entry->d_name, "..") == 0)
            continue;

        if (long_format)
            print_file_details(dirpath, entry->d_name);
        else
            printf("%s\n", entry->d_name);
    }

    closedir(dir);
    return 0;
```

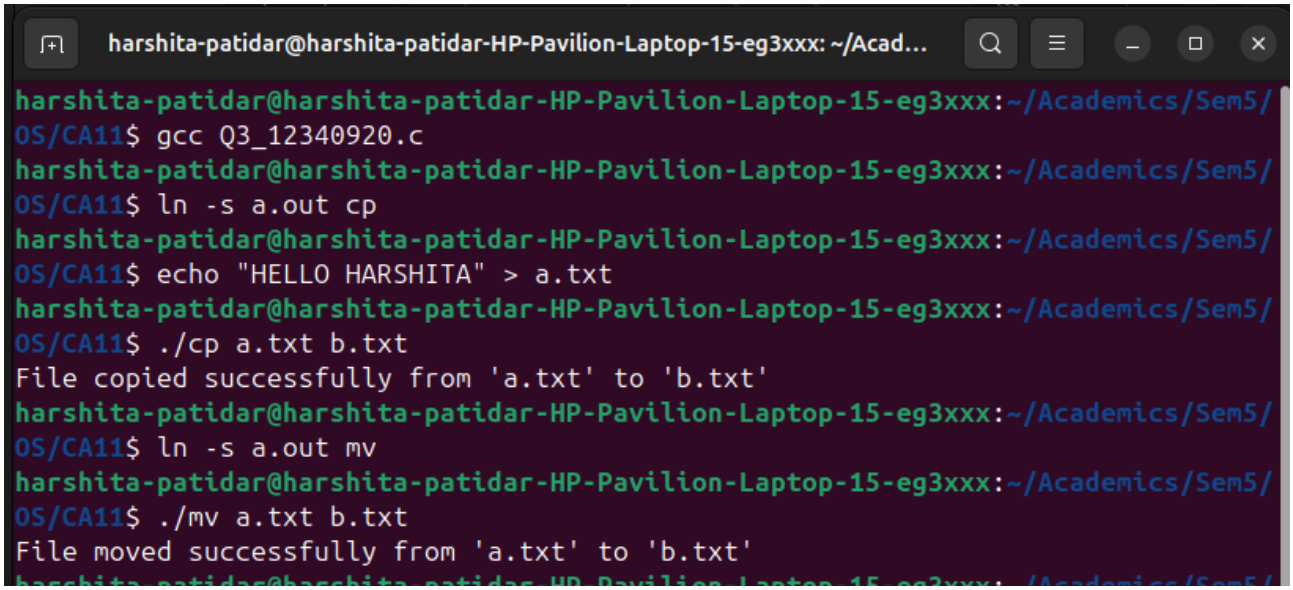
}

Screenshot:



Que 3:

Screenshot:



```
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx:~/Academics/Sem5/
OS/CA11$ echo "cmd cp" > c1.txt
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx:~/Academics/Sem5/
OS/CA11$ ./a.out cp c1.txt c2.txt
File copied successfully from 'c1.txt' to 'c2.txt'
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx:~/Academics/Sem5/
OS/CA11$ echo "cmd mv" > m1.txt
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx:~/Academics/Sem5/
OS/CA11$ ./a.out mv m1.txt m2.txt
File moved successfully from 'm1.txt' to 'm2.txt'
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx:~/Academics/Sem5/
OS/CA11$
```

CODE:

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/stat.h>
#include <string.h>
#include <errno.h>

#define BUF_SIZE 4096

int copy_file(const char *src, const char *dest) {
    int src_fd, dest_fd;
    struct stat src_stat;
    char buffer[BUF_SIZE];
    ssize_t bytes_read, bytes_written;

    src_fd = open(src, O_RDONLY);
    if (src_fd < 0) { perror("open source"); return -1; }

    if (stat(src, &src_stat) < 0) {
        perror("stat");
        close(src_fd);
        return -1;
    }

    dest_fd = open(dest, O_WRONLY | O_CREAT | O_TRUNC, src_stat.st_mode);
    if (dest_fd < 0) {
        perror("open destination");
        close(src_fd);
        return -1;
    }

     //ADD YOUR CODE HERE
    while ((bytes_read = read(src_fd, buffer, BUF_SIZE)) > 0) {
        bytes_written = write(dest_fd, buffer, bytes_read);
        if (bytes_written < 0) {
```

```
            perror("write");
            close(src_fd);
            close(dest_fd);
            return -1;
        }
    }


    if (bytes_read < 0) perror("read");

    close(src_fd);
    close(dest_fd);
    return 0;
}

int move_file(const char *src, const char *dest) {
    if (rename(src, dest) == 0) {
        return 0;   // Success
    }

    // rename failed: fallback to copy + unlink
    if (copy_file(src, dest) < 0) {
        return -1;  // Copy failed
    }

    if (unlink(src) < 0) {
        perror("unlink");
        return -1;
    }
    return 0;
}

int main(int argc, char *argv[]) {
    const char *prog;
    const char *src;
    const char *dest;


    if (argc == 3) {
         // ln -s a.out cp
         // ln -s a.out mv
         // ./cp src dest  or ./mv src dest
        prog = strrchr(argv[0], '/');
        prog = (prog == NULL) ? argv[0] : prog + 1;
        src = argv[1];
        dest = argv[2];
    } else if (argc == 4) {
        // ./a.out cp src dest  or ./a.out mv src dest
        prog = argv[1];
        src = argv[2];
        dest = argv[3];
    } else {
```

```
        fprintf(stderr, "Usage:\n");
        fprintf(stderr, "  %s <source> <destination>\n", argv[0]);
        fprintf(stderr, "  %s <cp|mv> <source> <destination>\n", argv[0]);
        exit(EXIT_FAILURE);
    }

    if (strcmp(prog, "cp") == 0) {
        if (copy_file(src, dest) == 0)
            printf("File copied successfully from '%s' to '%s'\n", src, dest);
        else
            fprintf(stderr, "Copy failed.\n");
    }
    else if (strcmp(prog, "mv") == 0) {
        if (move_file(src, dest) == 0)
            printf("File moved successfully from '%s' to '%s'\n", src, dest);
        else
            fprintf(stderr, "Move failed.\n");
    }
    else {
        fprintf(stderr, "Error: Invalid command name '%s'. Use cp or mv.\n", prog);
        exit(EXIT_FAILURE);
    }

    return 0;
}
```

QUE 4:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

#define MAX_INODES 8
#define MAX_BLOCKS 64
#define MAX_FILENAME 32
#define MAX_DESC 64
#define DIRECT_PTRS 4
#define MAX_DIR_ENTRIES 16

struct inode {
    int used;                   // 0 = free, 1 = allocated
    int size;                   // file size in bytes
    int type;                   // 0 = file, 1 = directory
    int creator_id;             // user ID who created file
    time_t created_at;          // creation timestamp
    char description[MAX_DESC];    // short file description
    int direct[DIRECT_PTRS];       // direct data block pointers
};
```

```
struct dir_entry {
    char name[MAX_FILENAME];
    int inum;
};

struct inode inode_table[MAX_INODES];
int data_block_bitmap[MAX_BLOCKS];   // 0 = free, 1 = used
struct dir_entry directory[MAX_DIR_ENTRIES];
int dir_count = 0;


int alloc_inode() {
    for (int i = 0; i < MAX_INODES; i++) {
        if (!inode_table[i].used) {
            inode_table[i].used = 1;
            inode_table[i].size = 0;
            inode_table[i].type = 0;
            inode_table[i].creator_id = 0;
            inode_table[i].created_at = time(NULL);
            strcpy(inode_table[i].description, "No description");
            for (int j = 0; j < DIRECT_PTRS; j++)
                inode_table[i].direct[j] = -1;
            return i;
        }
    }
    return -1;
}

void free_inode(int inum) {
    inode_table[inum].used = 0;
    inode_table[inum].size = 0;
    inode_table[inum].creator_id = 0;
    inode_table[inum].created_at = 0;
    strcpy(inode_table[inum].description, "");
    for (int j = 0; j < DIRECT_PTRS; j++)
        inode_table[inum].direct[j] = -1;
}

int alloc_block() {
    for (int i = 0; i < MAX_BLOCKS; i++) {
        if (data_block_bitmap[i] == 0) {
            data_block_bitmap[i] = 1;
            return i;
        }
    }
    return -1;
}

void free_block(int blocknum) {
    if (blocknum >= 0 && blocknum < MAX_BLOCKS)
        data_block_bitmap[blocknum] = 0;
```

```
}

int fs_create(const char *name, int creator_id, const char *desc) {
    for (int i = 0; i < dir_count; i++) {
        if (strcmp(directory[i].name, name) == 0) {
            printf("Error: File '%s' already exists\n", name);
            return -1;
        }
    }

    int inum = alloc_inode();
    if (inum < 0) {
        printf("Error: No free inode available\n");
        return -1;
    }


     // Fill metadata
     inode_table[inum].creator_id = creator_id;
     strncpy(inode_table[inum].description, desc, MAX_DESC - 1);
     inode_table[inum].description[MAX_DESC - 1] = '\0';
     inode_table[inum].created_at = time(NULL);

    int block = alloc_block();
    if (block < 0) {
        printf("Error: No free data block available\n");
        free_inode(inum);
        return -1;
    }

    inode_table[inum].direct[0] = block;
    inode_table[inum].size = 0;

    strcpy(directory[dir_count].name, name);
    directory[dir_count].inum = inum;
    dir_count++;

    printf("File '%s' created (inode %d, block %d)\n", name, inum, block);
    return inum;
}

int fs_delete(const char *name) {
    int found = -1;
    for (int i = 0; i < dir_count; i++) {
        if (strcmp(directory[i].name, name) == 0) {
            found = i;
            break;
        }
    }

    if (found < 0) {
        printf("Error: File '%s' not found\n", name);
```

```
        return -1;
    }

    int inum = directory[found].inum;

    for (int j = 0; j < DIRECT_PTRS; j++) {
        if (inode_table[inum].direct[j] != -1) {
        free_block(inode_table[inum].direct[j]);
        inode_table[inum].direct[j] = -1;
       }

    }

    free_inode(inum);

    for (int i = found; i < dir_count - 1; i++)
       directory[i] = directory[i + 1];
    dir_count--;

    printf("File '%s' deleted (inode %d freed)\n", name, inum);
    return 0;
}

void fs_show_state() {
    printf("\n--- File System State ---\n");
    printf("Directory entries (%d):\n", dir_count);
    for (int i = 0; i < dir_count; i++)
       printf("  %s → inode %d\n", directory[i].name, directory[i].inum);

    printf("\nInode table:\n");
    for (int i = 0; i < MAX_INODES; i++) {
       if (inode_table[i].used) {
          printf("  inode %d: creator=%d, size=%d, desc='%s'\n",
                i, inode_table[i].creator_id, inode_table[i].size, inode_table[i].description);
          printf("            created: %s", ctime(&inode_table[i].created_at));
          printf("            blocks: ");
          for (int j = 0; j < DIRECT_PTRS; j++)
             if (inode_table[i].direct[j] != -1)
                printf("%d ", inode_table[i].direct[j]);
          printf("\n");
       }
    }
    printf("--------------------------\n\n");
}


int main() {
    printf("File Creation and Deletion with Metadata Simulation\n");

    fs_create("file1.txt", 101, "User text document");
    fs_create("report.pdf", 102, "PDF project report");
    fs_create("data.bin", 103, "Binary data log");
```

```
    fs_show_state();

    fs_delete("report.pdf");
    fs_show_state();

    fs_create("notes.txt", 104, "Important notes");
    fs_show_state();

    return 0;
}
```

```
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx: ~/Academics/Sem5/OS/CA11
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx:~/Academics/Sem5/OS/CA11$ ./a.out
File Creation and Deletion with Metadata Simulation
File 'file1.txt' created (inode 0, block 0)
File 'report.pdf' created (inode 1, block 1)
File 'data.bin' created (inode 2, block 2)

--- File System State ---
Directory entries (3):
  file1.txt → inode 0
  report.pdf → inode 1
  data.bin → inode 2

Inode table:
  inode 0: creator=101, size=0, desc='User text document'
             created: Wed Nov 19 17:29:41 2025
             blocks: 0
  inode 1: creator=102, size=0, desc='PDF project report'
             created: Wed Nov 19 17:29:41 2025
             blocks: 1
  inode 2: creator=103, size=0, desc='Binary data log'
             created: Wed Nov 19 17:29:41 2025
             blocks: 2
-------------------------

File 'report.pdf' deleted (inode 1 freed)

--- File System State ---
Directory entries (2):
  file1.txt → inode 0
  data.bin → inode 2

Inode table:
  inode 0: creator=101, size=0, desc='User text document'
             created: Wed Nov 19 17:29:41 2025
             blocks: 0
  inode 2: creator=103, size=0, desc='Binary data log'
             created: Wed Nov 19 17:29:41 2025
             blocks: 2
-------------------------

File 'notes.txt' created (inode 1, block 1)
```

```
File 'notes.txt' created (inode 1, block 1)

--- File System State ---
Directory entries (3):
  file1.txt → inode 0
  data.bin → inode 2
  notes.txt → inode 1

Inode table:
  inode 0: creator=101, size=0, desc='User text document'
             created: Wed Nov 19 17:29:41 2025
             blocks: 0
  inode 1: creator=104, size=0, desc='Important notes'
             created: Wed Nov 19 17:29:41 2025
             blocks: 1
  inode 2: creator=103, size=0, desc='Binary data log'
             created: Wed Nov 19 17:29:41 2025
             blocks: 2
-------------------------

harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx:~/Academics/Sem5/OS/CA11$
```